**LABORATORY OBSERVATION / RECORD**

**Course:** Full Stack Web Development

**Course Code:** 23CM4121

**Experiment No.:** 3

**Student Name:** Kottana Roshini

**Roll No:A24126552263**

**1. TITLE**

Interactive To-Do List using DOM Manipulation and Event Listeners

**2. AIM**

To develop an interactive to-do list web application using JavaScript, DOM manipulation and event listeners, which allows a user to add tasks, mark them as completed, and delete them from the list.

**3. OBJECTIVE**

The objectives of this experiment are:

- To select and manipulate elements of the DOM.
- To dynamically create HTML elements using JavaScript.
- To dynamically modify the DOM at run time.
- To attach event listeners to elements.
- To handle click events for adding, completing and deleting tasks.
- To add and remove elements from the webpage.
- To build an interactive, event-driven webpage.

**4. THEORY**

The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify. Event listeners allow JavaScript to respond to user actions such as a button click.

In this application, the user types a task into a text box and clicks the “Add Task” button. This triggers a click event, inside which a new list item is created using document.createElement(), containing the task text along with a “Complete” button and a “Delete” button. The new item is then attached to the task list using appendChild().

Each task's Complete button has its own click listener that toggles a CSS class on the task, visually marking it as completed (strike-through text). Each task's Delete button has its own click listener that removes that specific task element from the DOM using remove(). When no tasks remain in the list, a “No tasks available.” message is displayed; the message is hidden as soon as at least one task exists.

The basic flow of the application is:

*User Input → Add Task Click Event → Task Element Created → Complete / Delete Listeners Attached → DOM Updated → Webpage Reflects Change*

**5. ALGORITHM / PROCEDURE**

- Create the project folder with index.html, style.css and script.js.
- Link the CSS and JavaScript files to the HTML page.
- Create a text input box and an “Add Task” button.
- Create an empty list container and an empty-list message paragraph.
- Select the input box, Add Task button, task list and empty message using document.getElementById().
- Write a function that shows the empty message only when the task list has no items.
- Add a click event listener to the Add Task button.
- Inside the listener, read and trim the value typed in the input box; ignore empty input.
- Dynamically create a list item containing the task text, a Complete button and a Delete button.
- Attach a click listener to the Complete button that toggles a “completed” class on the task.
- Attach a click listener to the Delete button that removes that task item from the DOM.
- Append the new task item to the task list and clear the input box.
- Update the empty-list message after every add and delete operation.
- Apply CSS styling to the input, buttons and task items, including a completed style.
- Open index.html in the browser and test adding, completing and deleting tasks.
- Perform multiple test cases and verify the actual output with the expected output.

**6. PROGRAM**

***index.html***

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Interactive To-Do List</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

    <h1>My To-Do List</h1>

    <div class="input-container">
        <input type="text" id="taskInput" placeholder="Enter a task...">
        <button id="addBtn">Add Task</button>
    </div>

    <ul id="taskList"></ul>

    <p id="emptyMessage">No tasks available.</p>

    <script src="script.js"></script>
</body>
</html>
```

***style.css***

```
body {
    font-family: Arial, sans-serif;
    background-color: #f4f6f8;
```

```
    text-align: center;
    padding: 30px;
}

.input-container {
    margin-bottom: 20px;
}

#taskInput {
    width: 280px;
    padding: 8px 10px;
    border: 1px solid #ccc;
    border-radius: 4px;
}

#addBtn {
    padding: 8px 16px;
    margin-left: 8px;
    background-color: #2e5496;
    color: #ffffff;
    border: none;
    border-radius: 4px;
    cursor: pointer;
}

#taskList {
    list-style: none;
    padding: 0;
```

```
    max-width: 420px;
    margin: 0 auto;
}

.task-item {
    display: flex;
    align-items: center;
    justify-content: space-between;
    background: #ffffff;
    padding: 10px 15px;
    margin-bottom: 10px;
    border-radius: 6px;
    box-shadow: 0 1px 4px rgba(0, 0, 0, 0.12);
}

.task-item.completed .task-text {
    text-decoration: line-through;
    color: #888888;
}

.complete-btn, .delete-btn {
    border: none;
    border-radius: 4px;
    padding: 6px 10px;
    margin-left: 8px;
    color: #ffffff;
    cursor: pointer;
}
```

```
.complete-btn { background-color: #2e9e5b; }
.delete-btn   { background-color: #d9534f; }

#emptyMessage {
   color: #888888;
   font-style: italic;
}
```

***script.js***

```
// DOM selection
const taskInput = document.getElementById("taskInput");
const addBtn = document.getElementById("addBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

// Show or hide the "No tasks available." message
function updateEmptyMessage() {
   emptyMessage.style.display =
      taskList.children.length === 0 ? "block" : "none";
}

// Add Task button click event
addBtn.addEventListener("click", () => {

   const taskText = taskInput.value.trim();
   if (taskText === "") {
      return;
   }
```

```
// Dynamically create a new task item
const li = document.createElement("li");
li.className = "task-item";

const span = document.createElement("span");
span.textContent = taskText;
span.className = "task-text";
li.appendChild(span);

// Complete button
const completeBtn = document.createElement("button");
completeBtn.textContent = "Complete";
completeBtn.className = "complete-btn";
completeBtn.addEventListener("click", () => {
  li.classList.toggle("completed");
});
li.appendChild(completeBtn);

// Delete button
const deleteBtn = document.createElement("button");
deleteBtn.textContent = "Delete";
deleteBtn.className = "delete-btn";
deleteBtn.addEventListener("click", () => {
  li.remove();
  updateEmptyMessage();
});
li.appendChild(deleteBtn);
```

```
    // Add the new task to the list
    taskList.appendChild(li);

    // Clear the input box
    taskInput.value = "";

    updateEmptyMessage();
});

// Show the empty message on first load
updateEmptyMessage();
```

**7. CODE EXPLANATION**

| Code | Explanation |
|---|---|
| document.getElementById() | Selects an existing element from the DOM. |
| addEventListener("click", ...) | Runs a function whenever the element is clicked. |
| .value.trim() | Reads the task text and removes extra spaces. |
| document.createElement() | Dynamically creates a new HTML element. |
| .textContent | Sets the visible text inside a created element. |
| .className | Assigns a CSS class to a created element. |
| .appendChild() | Adds a newly created element to the webpage. |
| .classList.toggle("completed") | Adds or removes the completed styling on a task. |
| .remove() | Removes a specific task element from the DOM. |
| taskList.children.length | Counts how many tasks currently exist. |

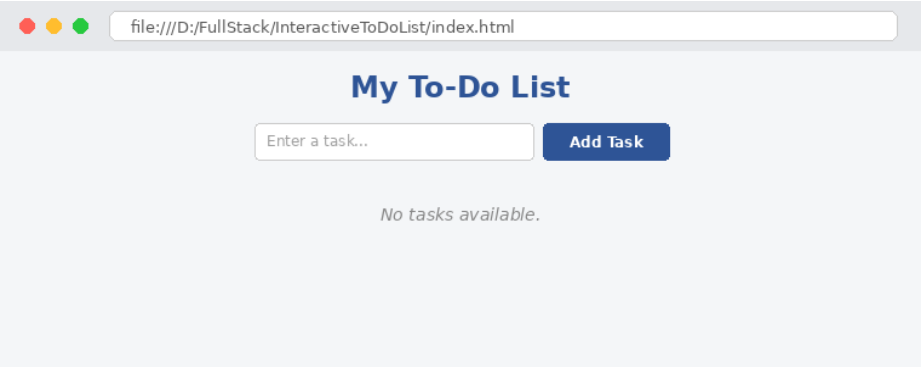
| Code | Explanation |
|---|---|
| .style.display | Shows or hides the empty-list message. |

**8. TEST CASES AND EXECUTION DATA**

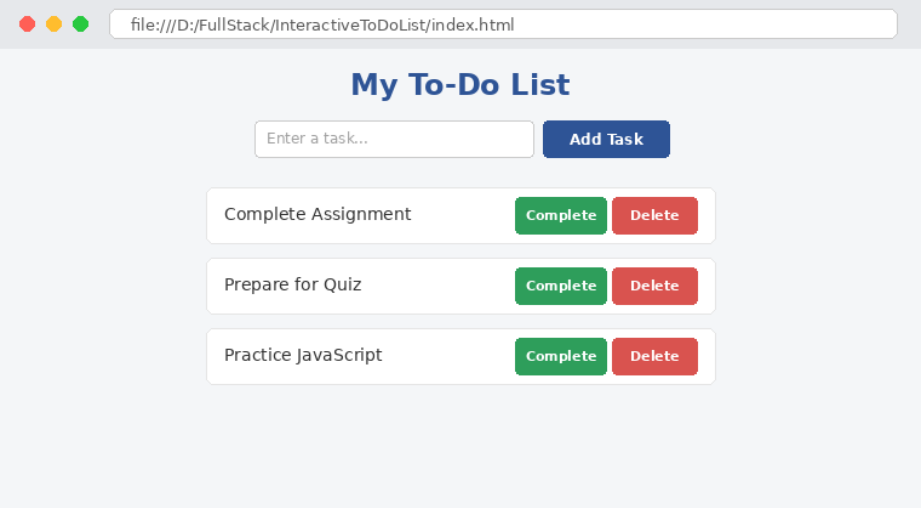
| Test Case | Action | Expected Result | Status |
|---|---|---|---|
| TC-01 | Click "Add Task" with input empty | No task is added; list remains unchanged | Pass |
| TC-02 | Type "Complete Assignment" and click Add Task | Task appears in the list with Complete and Delete buttons | Pass |
| TC-03 | Add "Prepare for Quiz" and "Practice JavaScript" | Both tasks appear below the first task | Pass |
| TC-04 | Click "Complete" on "Complete Assignment" | Task text shows a strike-through style | Pass |
| TC-05 | Click "Complete" again on the same task | Strike-through style is removed | Pass |
| TC-06 | Click "Delete" on "Prepare for Quiz" | Task is removed from the list immediately | Pass |
| TC-07 | Delete all remaining tasks | "No tasks available." message is displayed | Pass |

**9. OUTPUT**

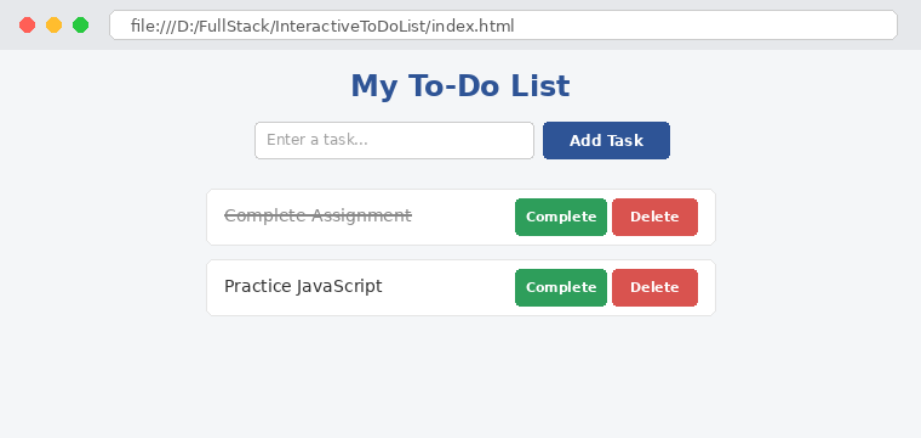
**Output 1: Before adding any tasks**




**Output 2: After adding three tasks**




**Output 3: After completing "Complete Assignment" and deleting "Prepare for Quiz"**




**10. RESULT**

Thus, an Interactive To-Do List application was successfully developed using JavaScript, DOM manipulation and event listeners. Tasks were dynamically added, marked as completed, and deleted from the webpage, and an appropriate message was displayed when the list was empty. The application was verified using multiple test cases.